# YANGを中心とした SDNの開発ワークフロー

ソフトバンク株式会社
コア&トランスポート技術本部
C&I統括部 パケットネットワーク部
堀場 勝広

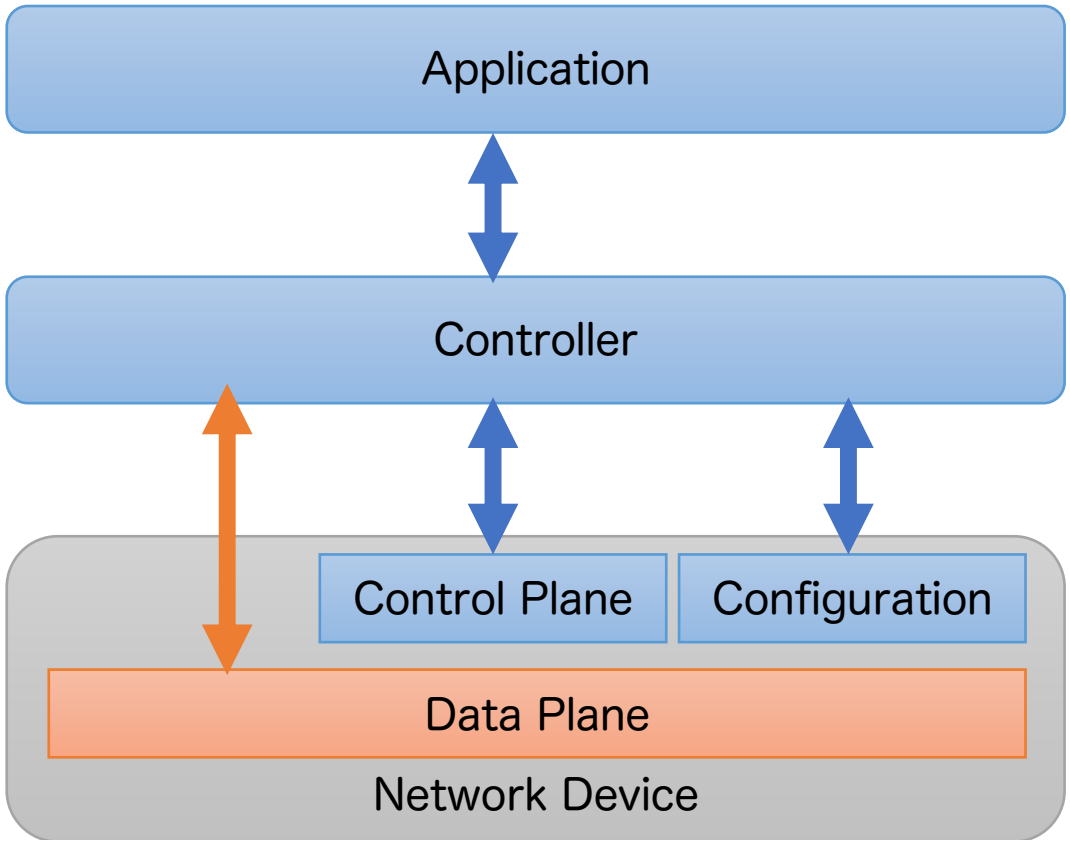
# 概要

- 自己紹介
- ネットワークプログラマビリティとWebの技術
- Webの技術者たちが利用している道具
- YANGによるネットワークとWebの技術の紐付け
- 最近の動向
- まとめ

# 自己紹介

- 元大学研究者
  - 研究領域はNW仮想化、SDN、NFVなど
  - コンソーシアムなどで企業の皆さんとオープンソース技術を研究
- 2015年からソフトバンク株式会社
  - オープンソース技術を利用したSDNの研究開発に従事
  - OpenDaylightとの出会い
  - Lithium(3rd Release)から参戦
- 2017年から商用ネットワーク設計部門に異動
  - OpenDaylightを利用した商用ネットワークのSDN化に従事
  - SDNコントローラの上位システム(自動化システム)を含む

# ネットワークプログラマビリティと データプレーンプログラマビリティ




ネットワーク
プログラマビリティ

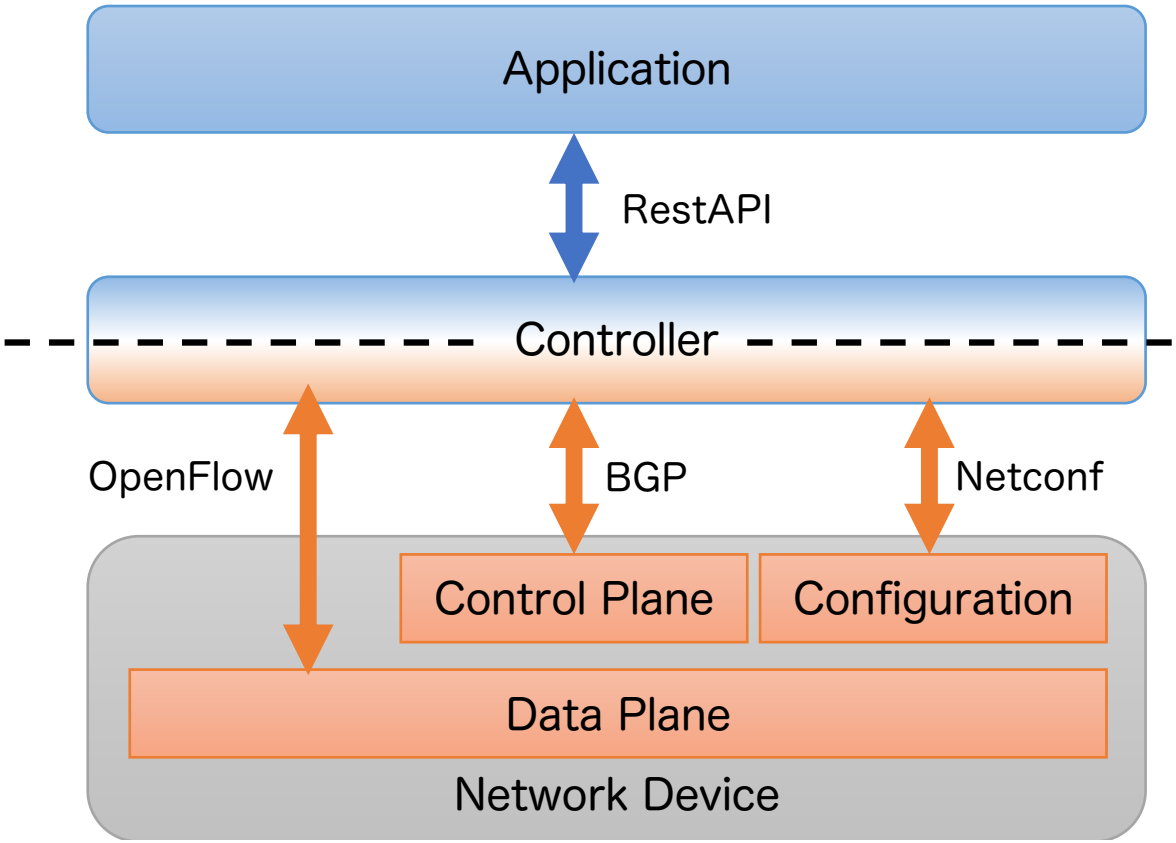
自動化(非CLIな設定管理)
Software Defined(SDN)

データプレーン
プログラマビリティ

パケット処理

# 今日のトピック

## いかにネットワーク屋の技術をWeb屋のそれに近づけるか




**Web屋さんの技術**

巨大データベースに対し
非同期にアクセスする
ステートレスな世界

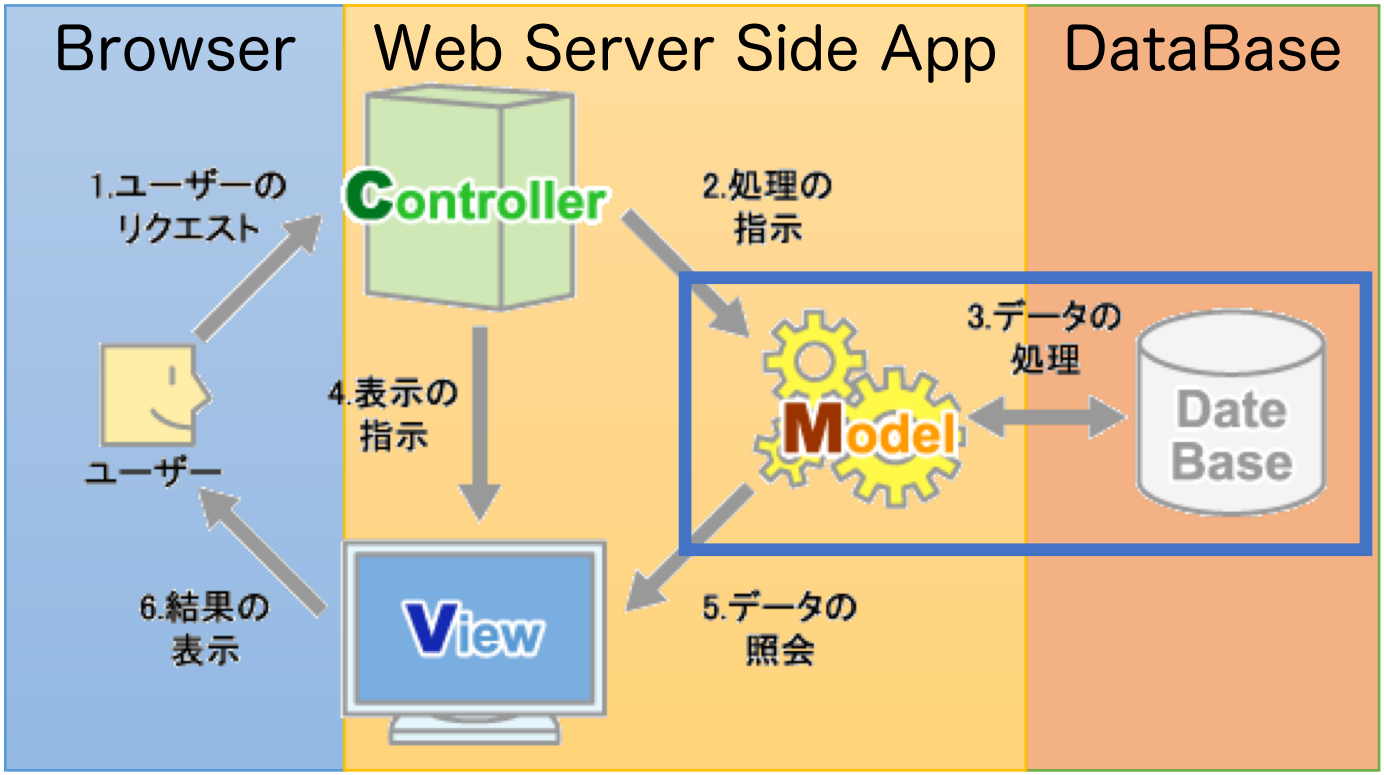
**ネットワーク屋さんの技術**

経路制御プロトコルや
パケット転送テーブルなど
ステートフルな世界

# Webの技術者たちが利用している道具

# モダンなWebアプリのフレームワーク

## Model View Controller(MVC)パターンで成り立っている




## Point

データの読み書きで
DBには直接アクセ
スしない

必ずモデルを介して
アクセスする

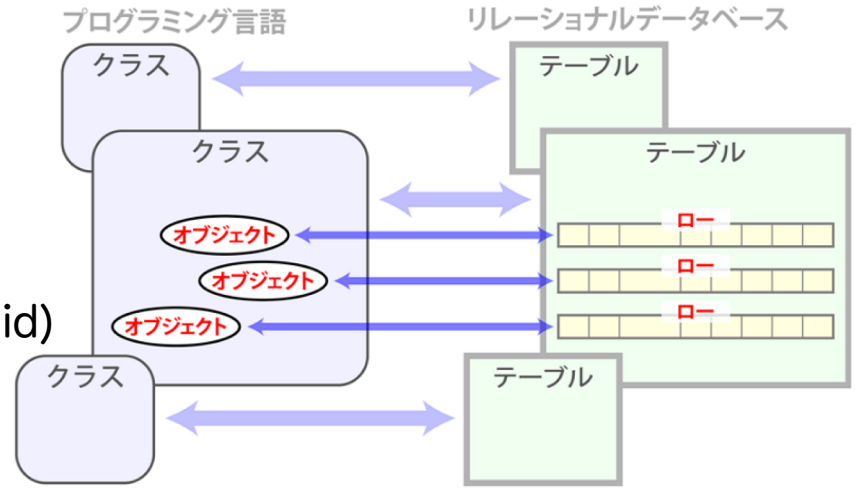
軽量なMVCフレームワークの自作
https://codezine.jp/article/detail/104

# オブジェクト関係マッピング
## ORM(Object Relational Mapping)

DBを直接操作するSQLをプログラミングするのは大変
できることならオブジェクト指向でプログラミングしたい
→ORMでデータベースとオブジェクトのギャップを埋める

```
import Employee

uid = 1608096
employee = new Employee(uid)
print employee.getName()
```

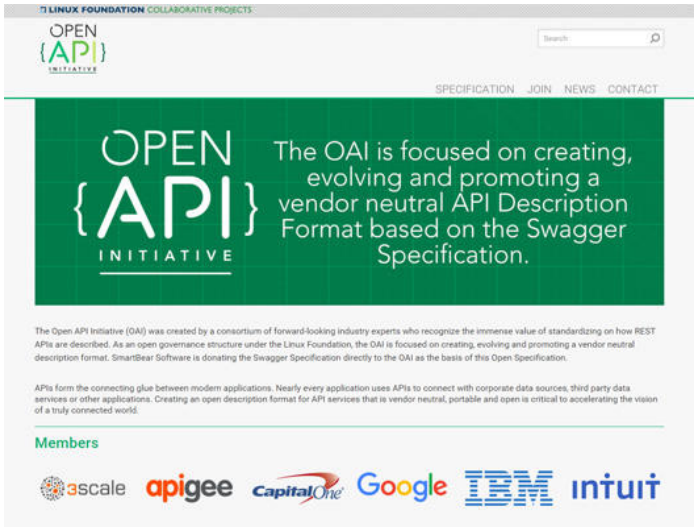



```
conn = connect(db,u, p)
cur = conn.cursor()
sql = “select * from”
table = “employee”
uid = “1608096”
sql += table
sql += “where uid=”
sql += uid
result = cur.exec(sql)
print result[2]
```

IPAセキュアプログラミング O/Rマッパ
https://www.ipa.go.jp/security/awareness/vendor/programmingv2/contents/004.html

# OPEN APIとSwagger

REST APIの標準フォーマット策定




https://openapis.org/

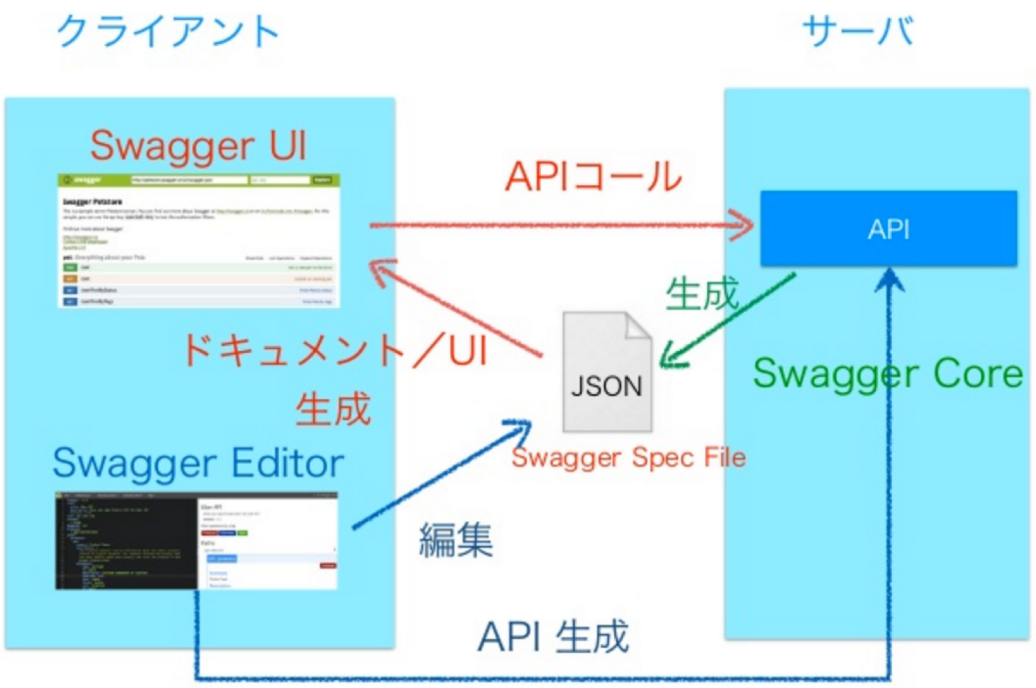
標準化フォーマット記述された
APIの利活用フレームワーク

プログラミング言語非依存な
REST APIインターフェース




http://swagger.io/

# Swaggerの概念




API仕様の仕様をYAMLで記述する
(Swagger Editor)

JSON形式の仕様(Swagger Spec)
に変換する

ドキュメント・UI(サンドボックス)
を自動生成(Swagger UI)

クライアントのコードも
自動生成(Swagger Codegen)

**Swaggerとは何か？**
https://blog.takuros.net/entry/2015/12/02/082248

# ここまでのまとめ

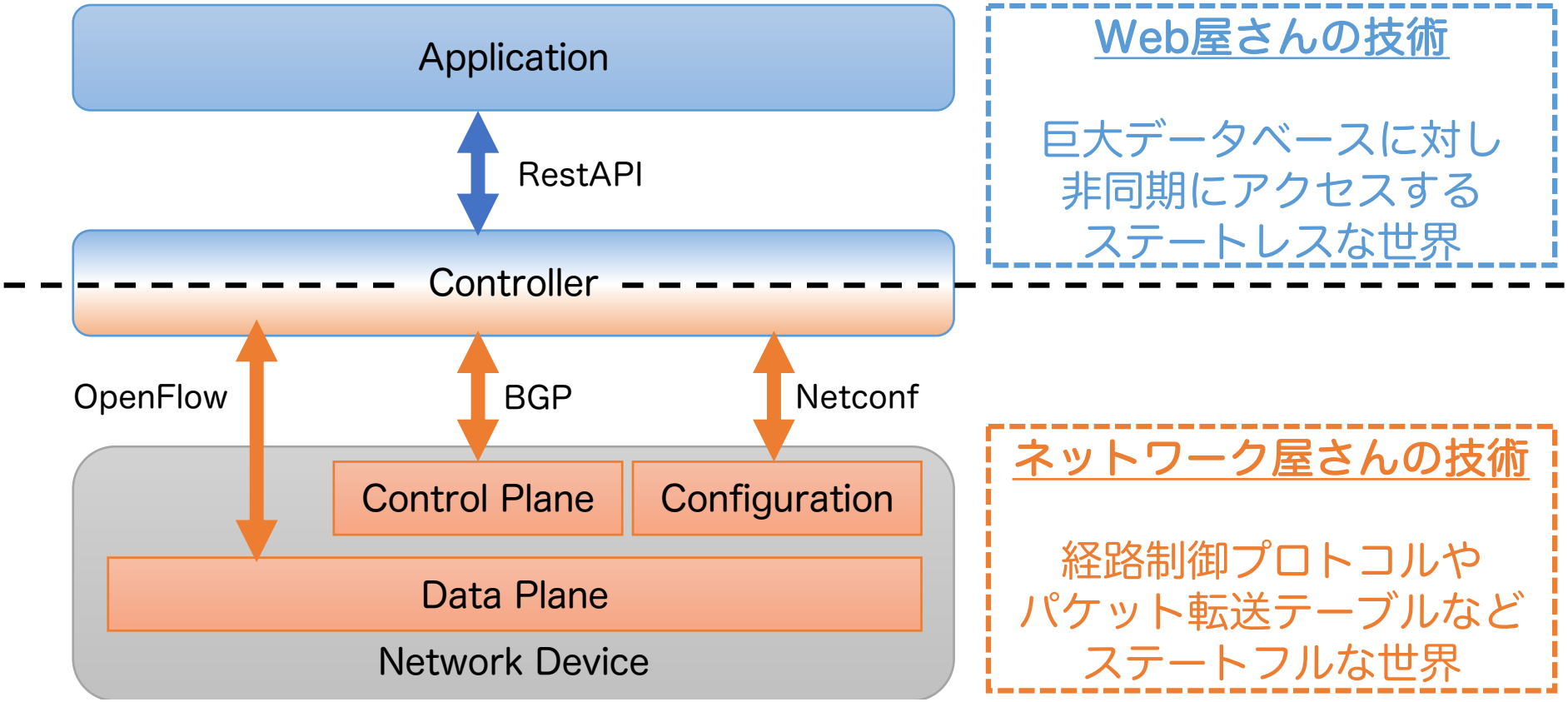
- データベースへのアクセス
  - 必ずモデルを通す
  - ORMを通してプログラミングする

- Restful API仕様と実装
  - 仕様をOpen APIで記述する
  - ドキュメント、サンドボックスはSwaggerで自動生成
  - クライアントのコードもSwaggerで自動生成

# YANGによるネットワークとWebの技術の紐付け

# 今日のトピック(再掲)

いかにネットワーク屋の技術をWeb屋のそれに近づけるか



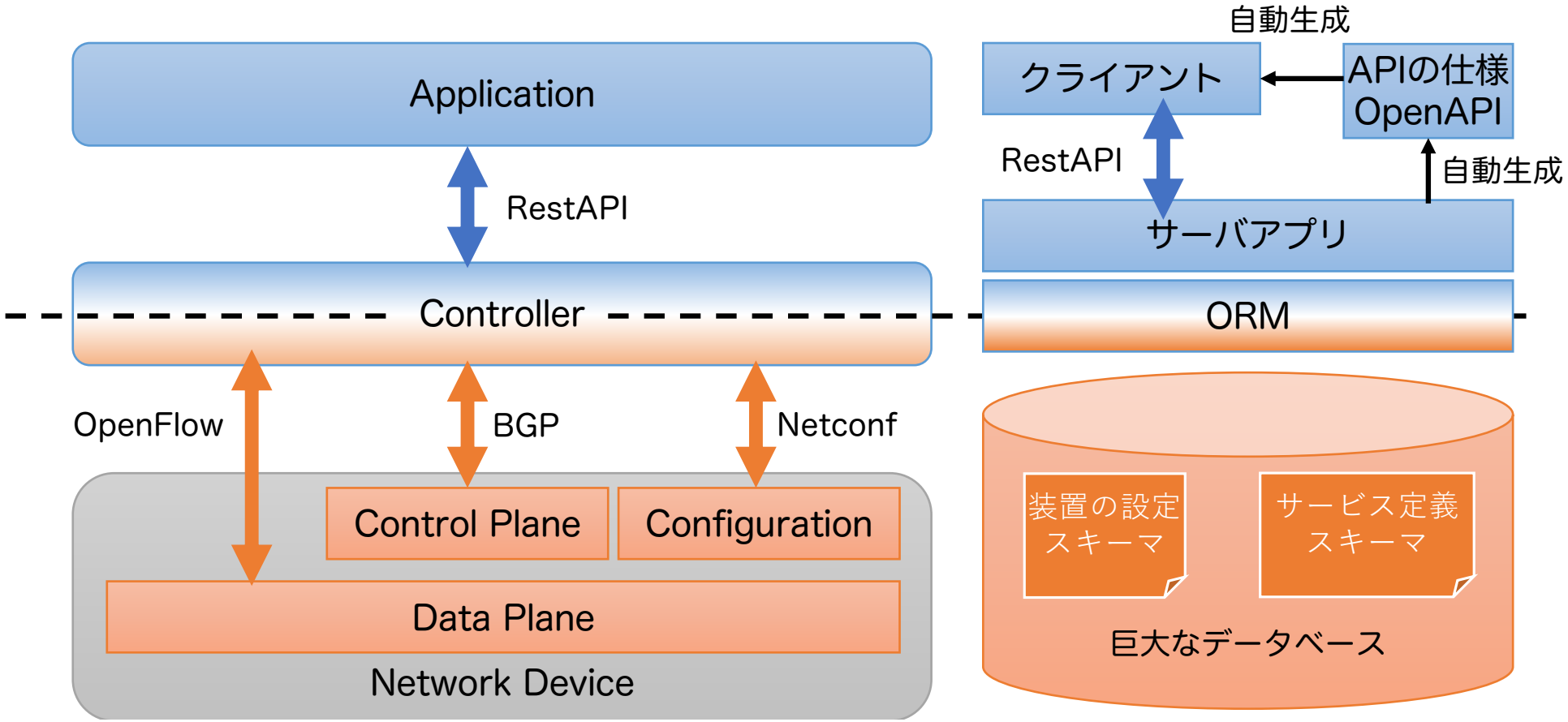

**Web屋さんの技術**

巨大データベースに対し
非同期にアクセスする
ステートレスな世界

**ネットワーク屋さんの技術**

経路制御プロトコルや
パケット転送テーブルなど
ステートフルな世界

# こうなれば幸せな世界


自動生成
Application
クライアント
APIの仕様
OpenAPI
RestAPI
RestAPI
自動生成
サーバアプリ
Controller
ORM
OpenFlow
BGP
Netconf
Control Plane
Configuration
装置の設定
スキーマ
サービス定義
スキーマ
Data Plane
巨大なデータベース
Network Device

そこでYANGの出番です

# YANG

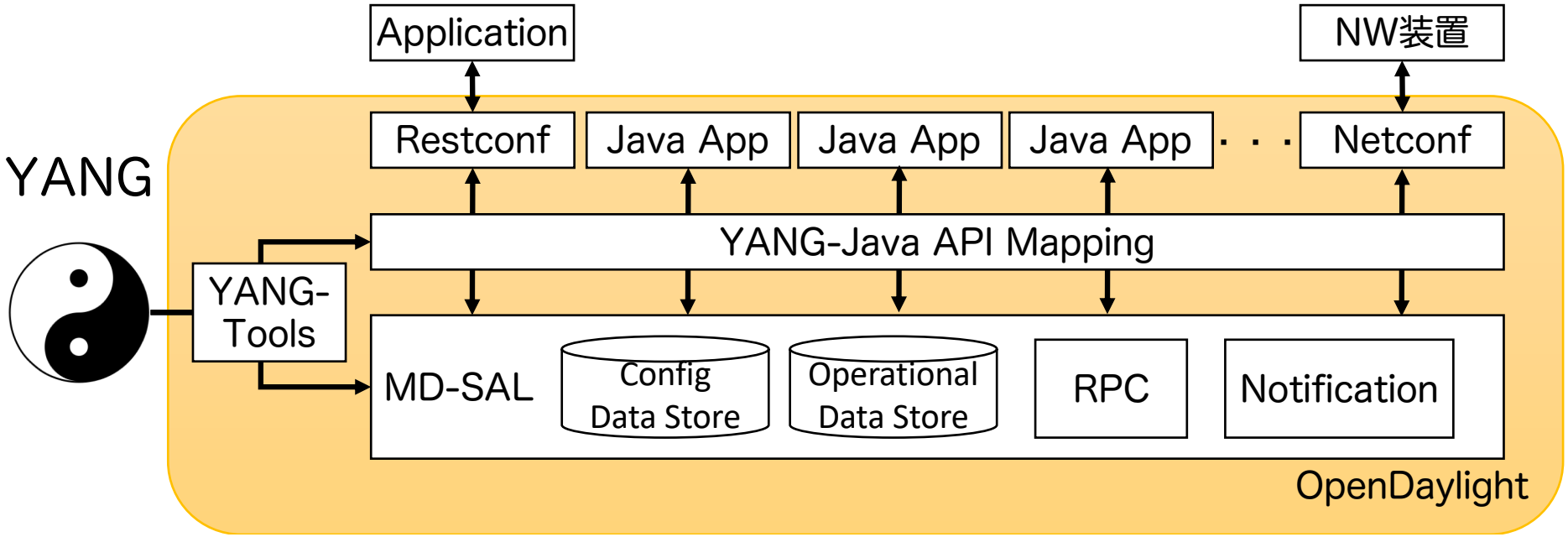
*YANG is a data modeling language used to **model configuration data, state data, Remote Procedure Calls, and notifications** for network management protocols.*
*/SNIP/*
*This document also specifies **the YANG mappings to the Network Configuration Protocol (NETCONF).***

**RFC7950 The YANG 1.1 Data Modeling Language**
https://tools.ietf.org/html/rfc7950

- YANGはあくまで「データモデル言語」
- 装置の設定以外に、サービスやリソースの定義などにも利用可能
  - 装置ベンダーにおける設定文法
  - IETFにおけるネットワークサービスやトポロジの定義
- Netconfにおける装置の設定や状態の文法は適用例の一つ
  - Restconfなどにも利用されている

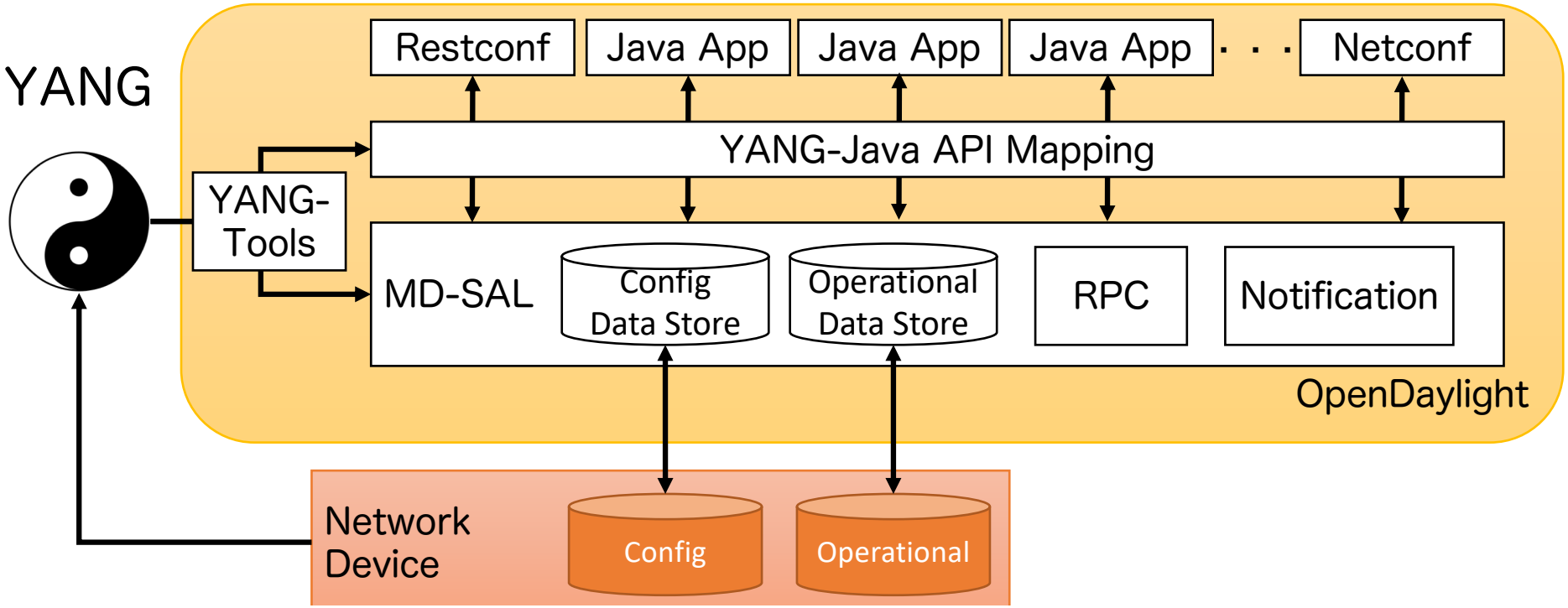
# OpenDaylight MD-SALとYANG

- MD-SAL(Model Driven Service Abstraction Layer)
- YANGを入力するとMD-SALへのAPIがORMとして自動生成される
- 実はNorthboundもSouthboundもMD-SALを利用する1つのアプリ



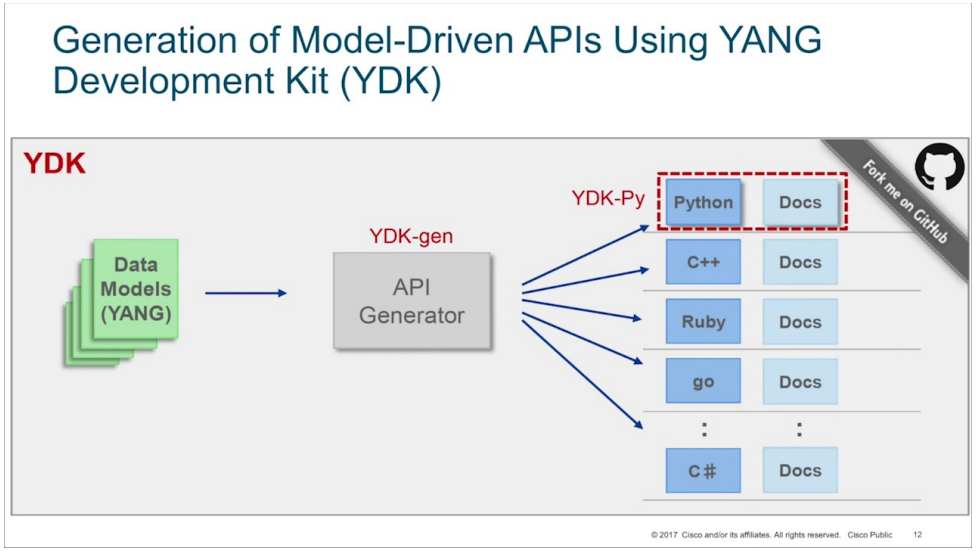
# YANG Mountと言う概念

- UNIXのファイルシステムと同じ概念
- MD-SALのデータストアに、装置の設定・状態のデータストアを連結

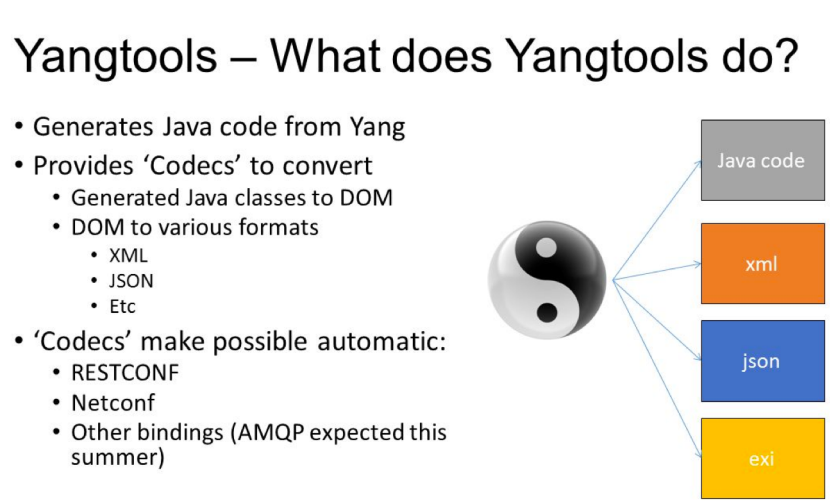


# YANGは様々な言語のORMに変換可能

## モダンなWebフレームワークの世界へ僕らも行ける！



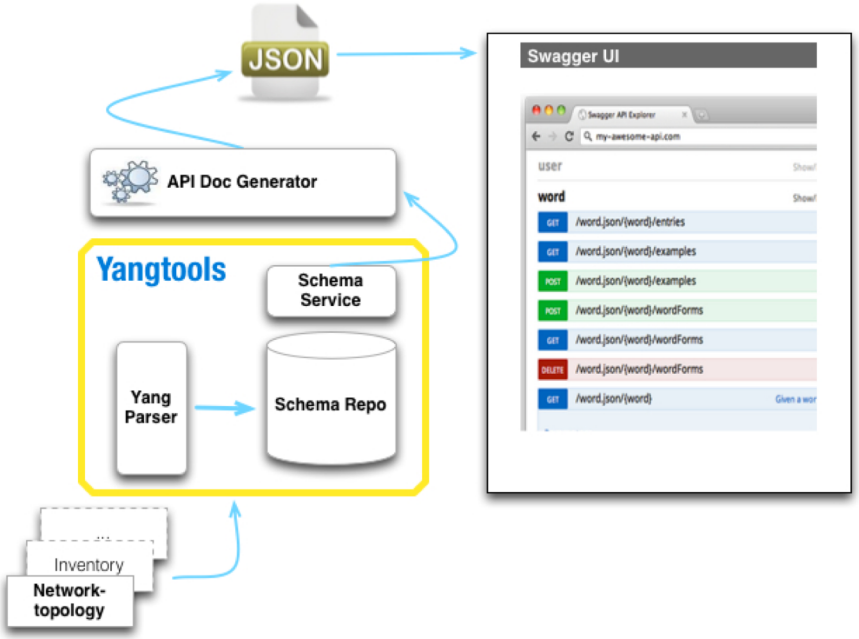

Infrastructure as a Code Using YANG, OpenConfig and YDK
https://www.youtube.com/watch?v=G1b6vJW1R5w




OpenDaylight Architecture
https://slideplayer.com/slide/5876657/

# YANGはSwagger Specにも変換可能

## クライアントやサンドボックスは自動生成する時代




OpenDaylightは、YANGを取り込んだ瞬間から、サンドボックスもできる

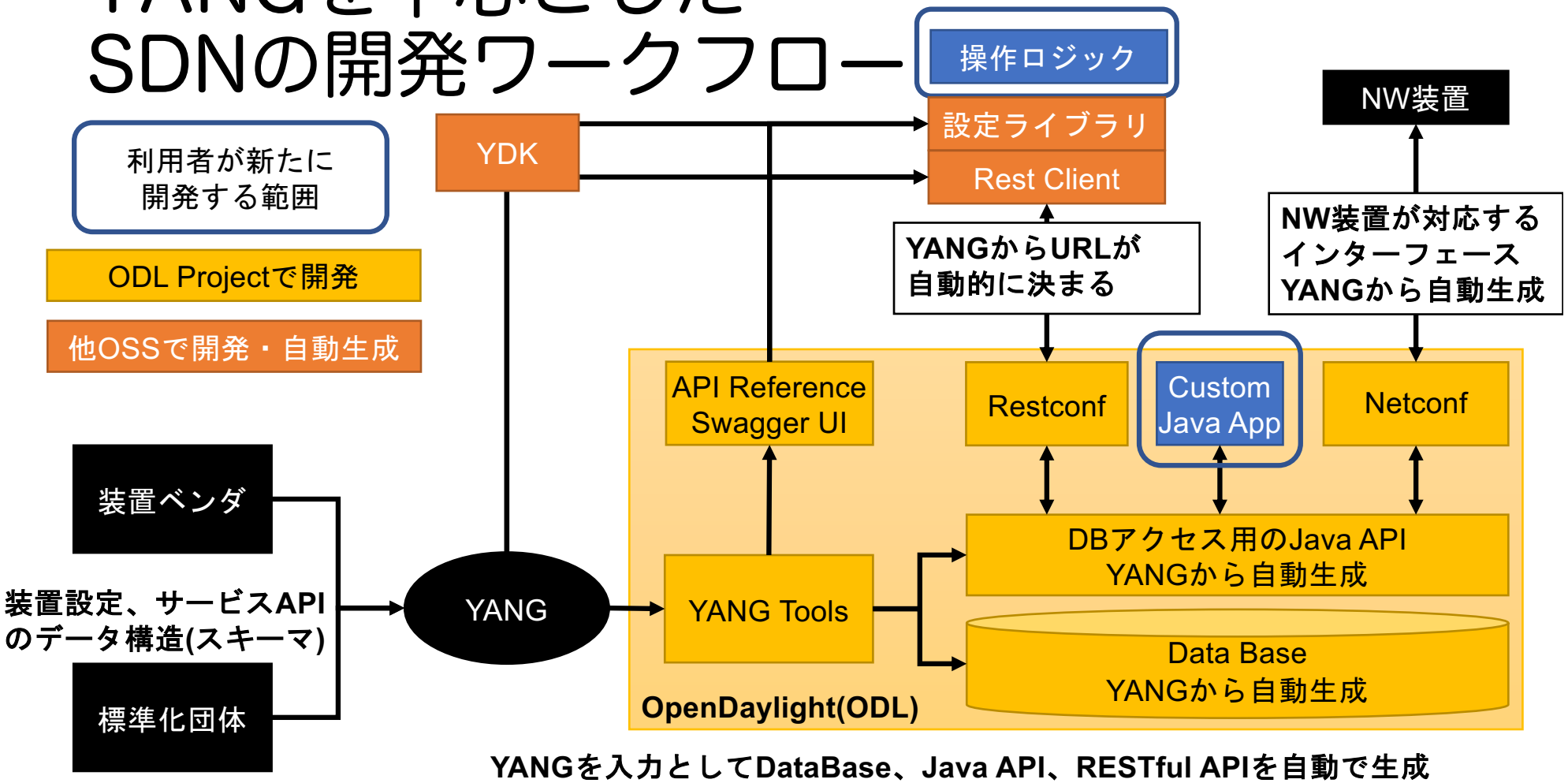
その他、変換ツールも公開されている

yang2swagger
- https://bitbucket.org/cmurch/yang2swagger-generator

yang-swagger
- https://github.com/corenova/yang-swagger
- https://www.npmjs.com/package/yang-swagger

**OpenDaylight Controller:MD-SAL:Restconf API Explorer**
https://wiki.opendaylight.org/view/OpenDaylight_Controller:MD-SAL:Restconf_APIExplorer

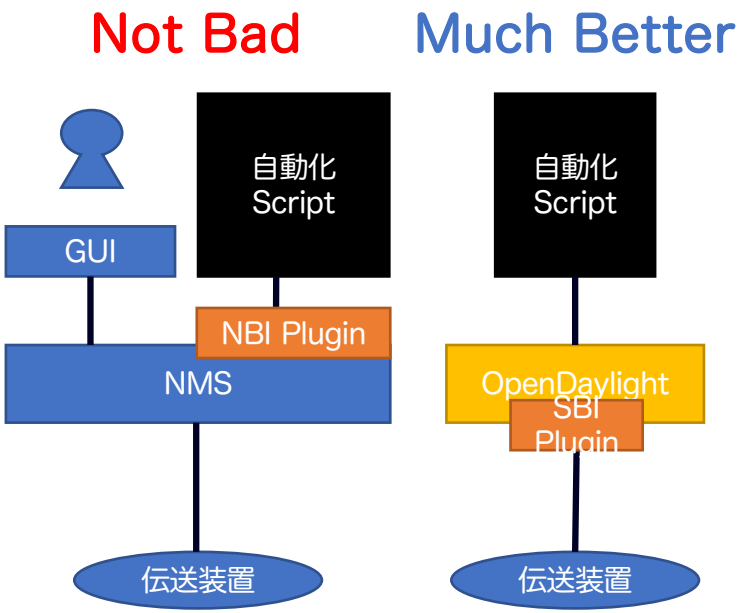
# YANGを中心とした SDNの開発ワークフロー




YANGを入力としてDataBase、Java API、RESTful APIを自動で生成

# レガシー装置対応の例

諦め掛けていた伝送系のレガシー装置も対応可能



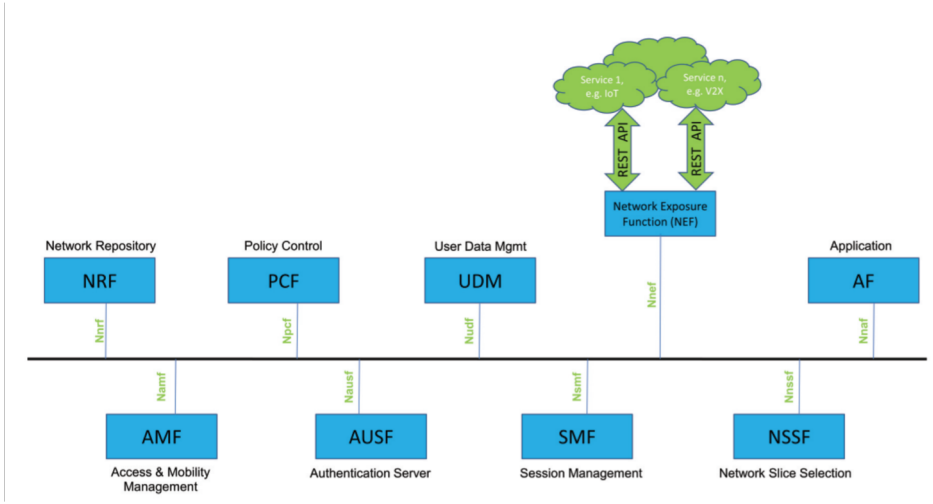

- とある伝送装置の話・・・
  - NMSとセットの製品
  - SBIのトランスポートはITU-T勧告ベース
  - 装置の設定データベースはSQL実装
- あれ、思ったより簡単だった・・・
  - SQLスキーマからYANGスキーマに変換
  - YANGをOpenDaylightにインポート
  - トランスポートを実装
  - 以上、終了・・・
- NMSにNorthbound APIを実装より楽！

# 最近の動向

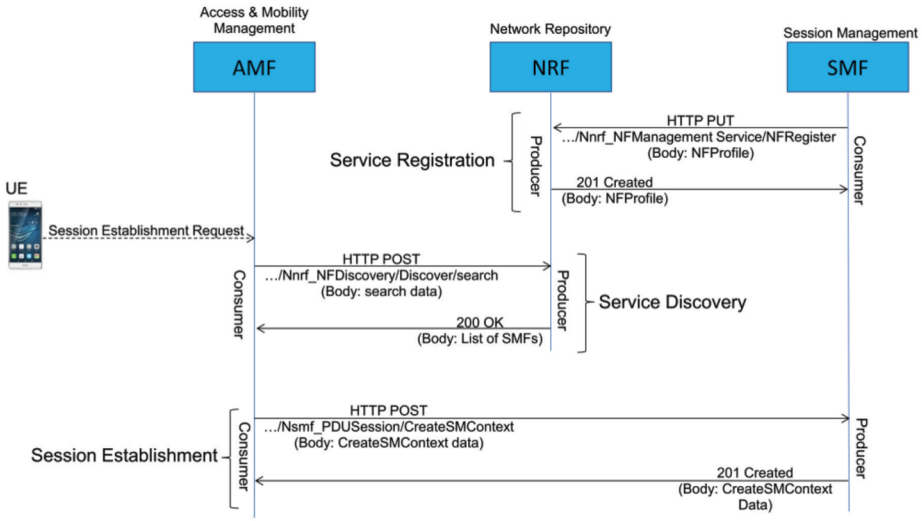
# 最近の動向(YANG)

- YANGはGithubで公開されている
  - 装置ベンダーさん、IETFで標準化されているYANG
    - https://github.com/YangModels/yang/tree/master/vendor
- 装置に非依存な設定も続々と定義されている
  - OpenConfig
    - https://github.com/openconfig/public
  - BroadBand Forum
    - https://github.com/BroadbandForum/yang
- ソフトウェアルータもYANG対応に舵を切った
  - FRR Advanced Northbound API
    - https://www.dropbox.com/s/zhybthruwocbqaw/netdef-frr-northbound.pdf

# 5G CoreもRestful APIの世界




RESTful APIs for the service based interfaces and northbound communication.

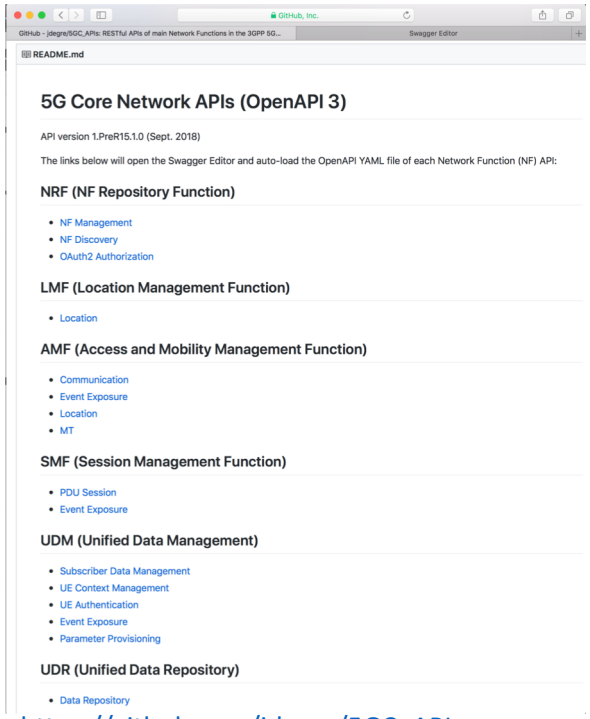



Simplified API calls for example SBA procedures

RESTful APIs for the 5G Service Based Architecture
https://www.riverpublishers.com/journal/journal_articles/RP_Journal_2245-800X_617.pdf

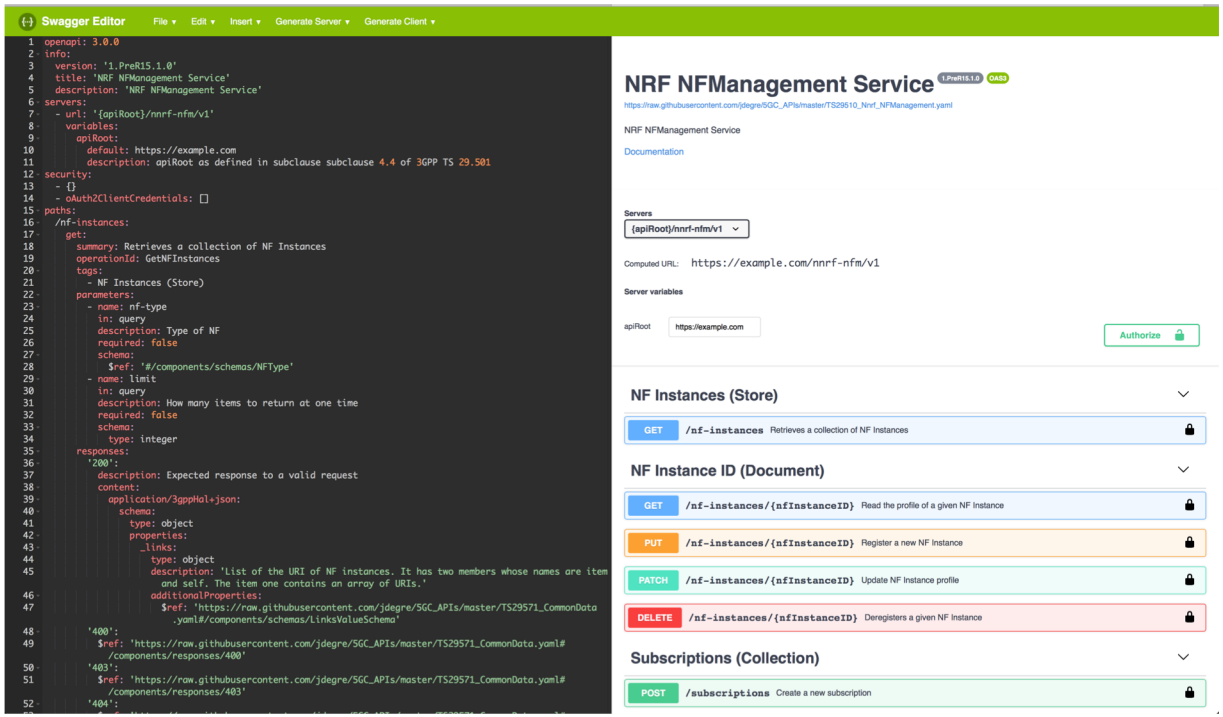
# 5G CoreのAPI定義

## 全てのAPIをGithubで公開
## OpenAPI(Swagger YAML)形式




https://github.com/jdegre/5GC_APIs

## 自然とSwaggerで編集可能
## もちろんサンドボックスやコードも自動生成可能




https://editor.swagger.io/?url=https://raw.githubusercontent.com/jdegre/5GC_APIs/master/TS29551_Nnef_PFDmanagement.yaml

# まとめ

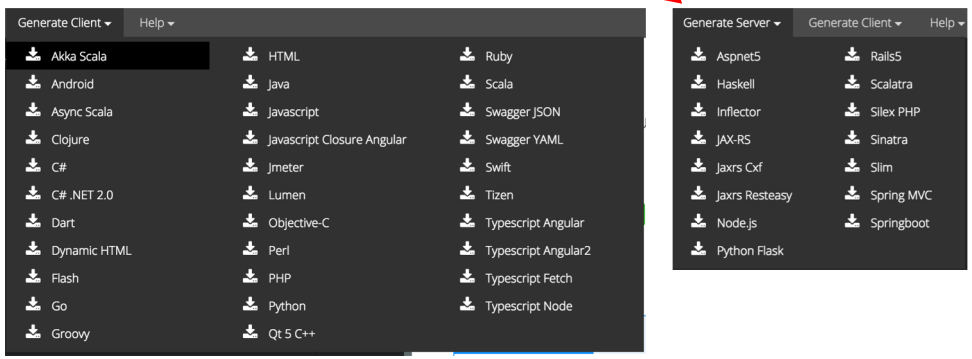
- ネットワークプログラマビリティ向上にはWebの技術が不可欠
- Webの技術者たちが利用している道具
  - Model View Controllerフレームワーク
  - Object Relational Mapping
  - Open APIとSwagger
- YANGによるネットワークとWebの技術の紐付け
  - YANGはあくまでデータモデルの定義
  - YANGをOpenDaylightのMD-SALに入力すれば概ね解決する
  - YANGはSwaggerとの親和性も高い

Thank You
May the YANG be with you.

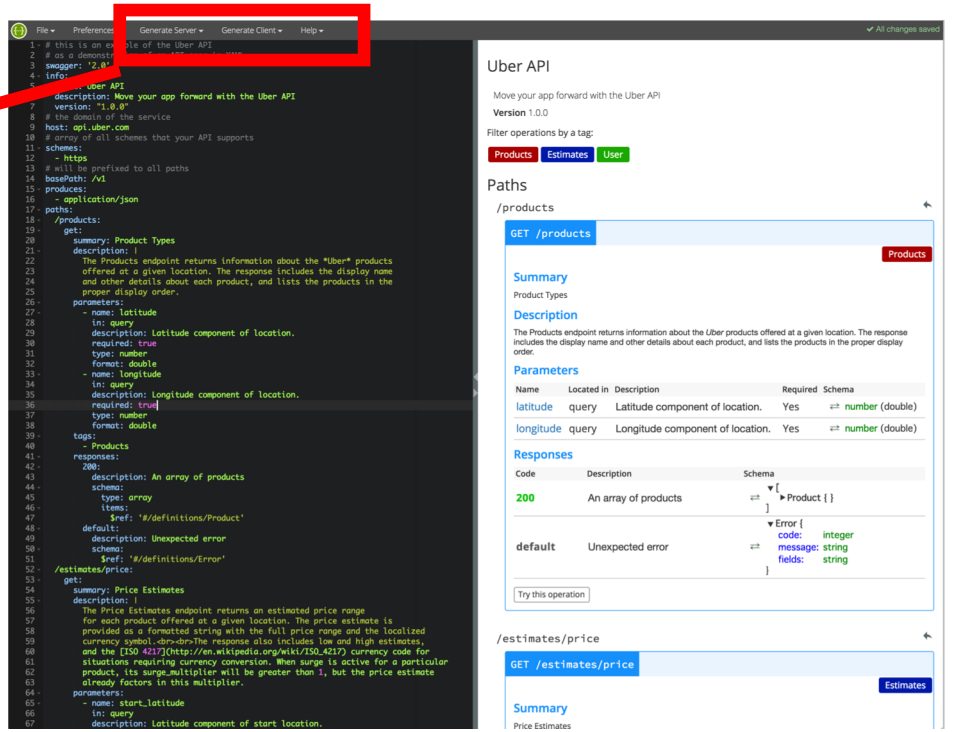
# Appendix

# Swagger Editor

標準フォーマット (YAML形式)に従って，Swagger Spec(API仕様)を記述
→ ドキュメント，クライアント，サーバのコードを自動生成




自動生成されるサーバとクライアントの言語




http://editor.swagger.io/

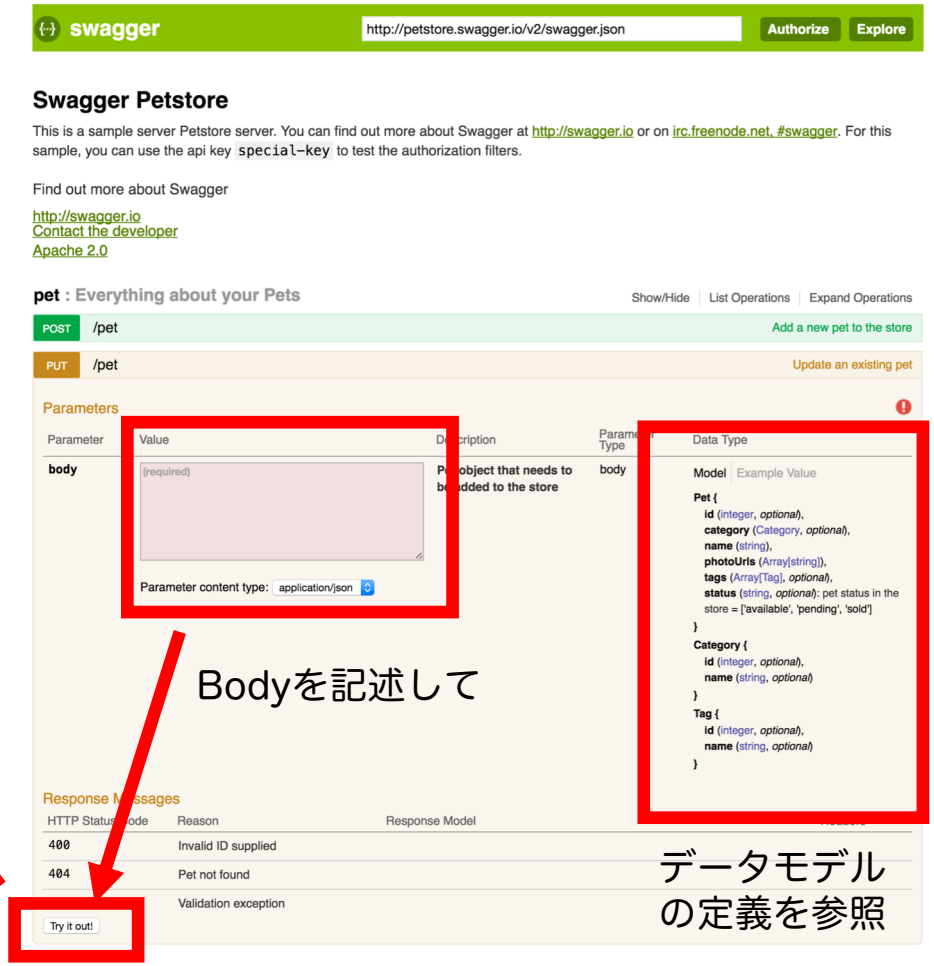
# Swagger UI

ドキュメント，テスト環境は
Swagger Specから自動生成

http://petstore.swagger.io/

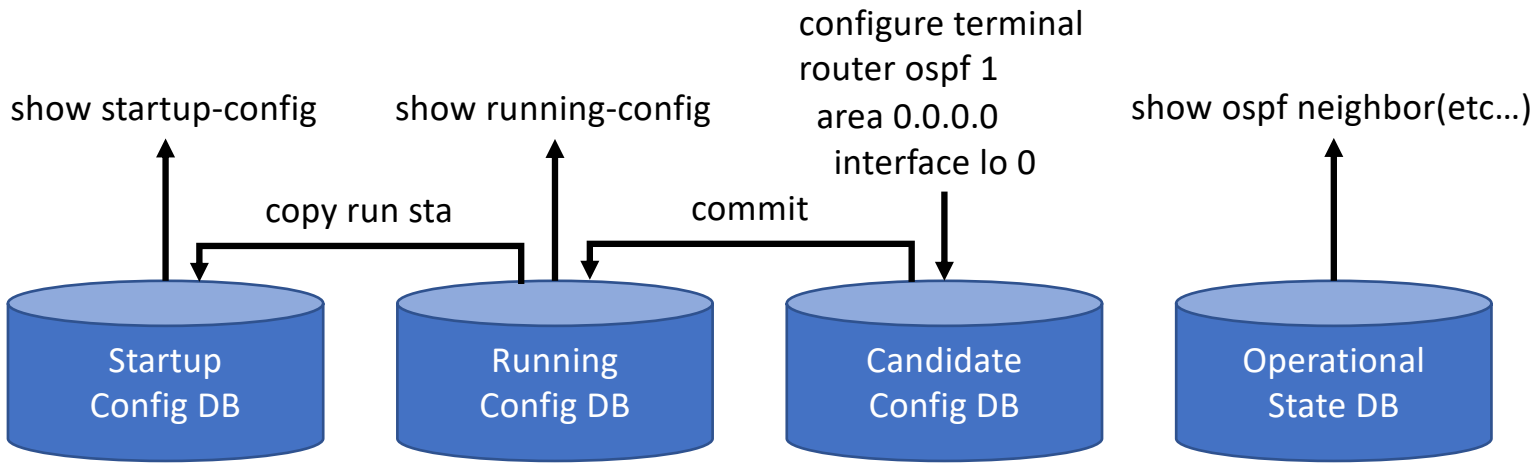


その場でAPIテスト
可能なSandbox



# モダンなNW装置はデータベースである

## 装置の設定、状態などはデータベースに格納されている




起動時に参照される設定の格納場所

動作中設定の格納場所
show runで参照される

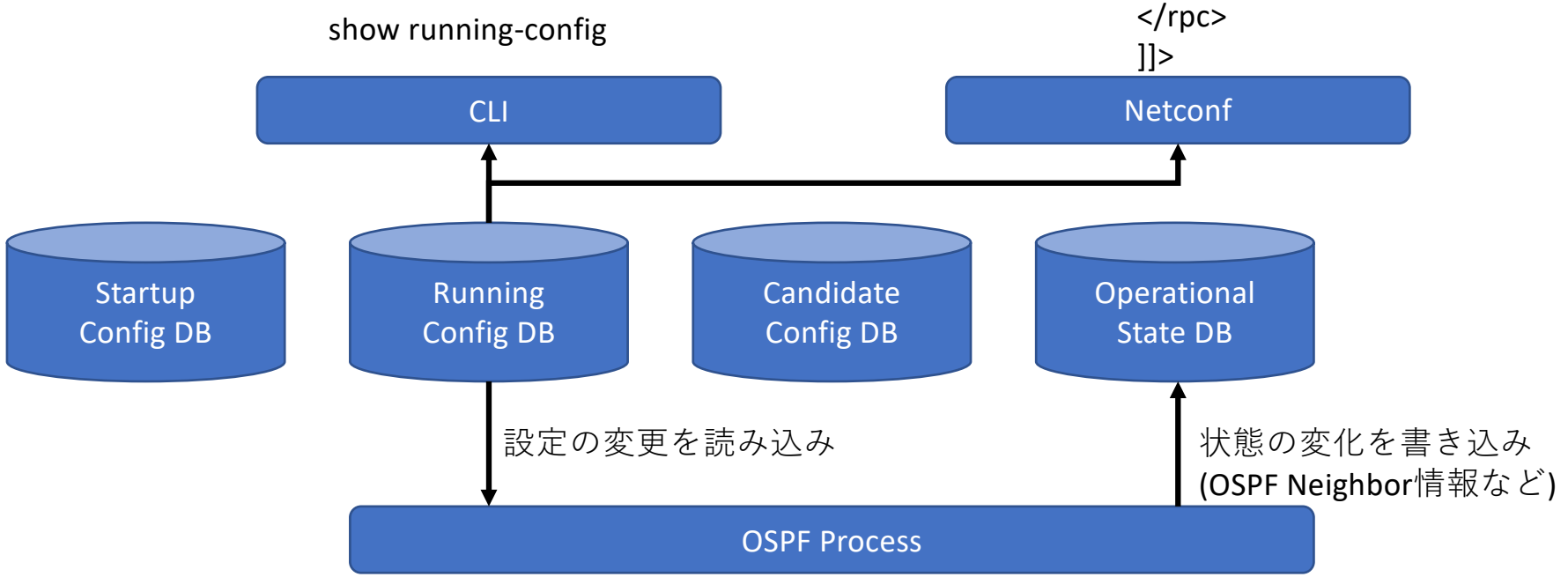
Commitする前の設定の格納場所

show xxxで参照されるネットワークの状態

# モダンなNW装置の仕組み(contd.)

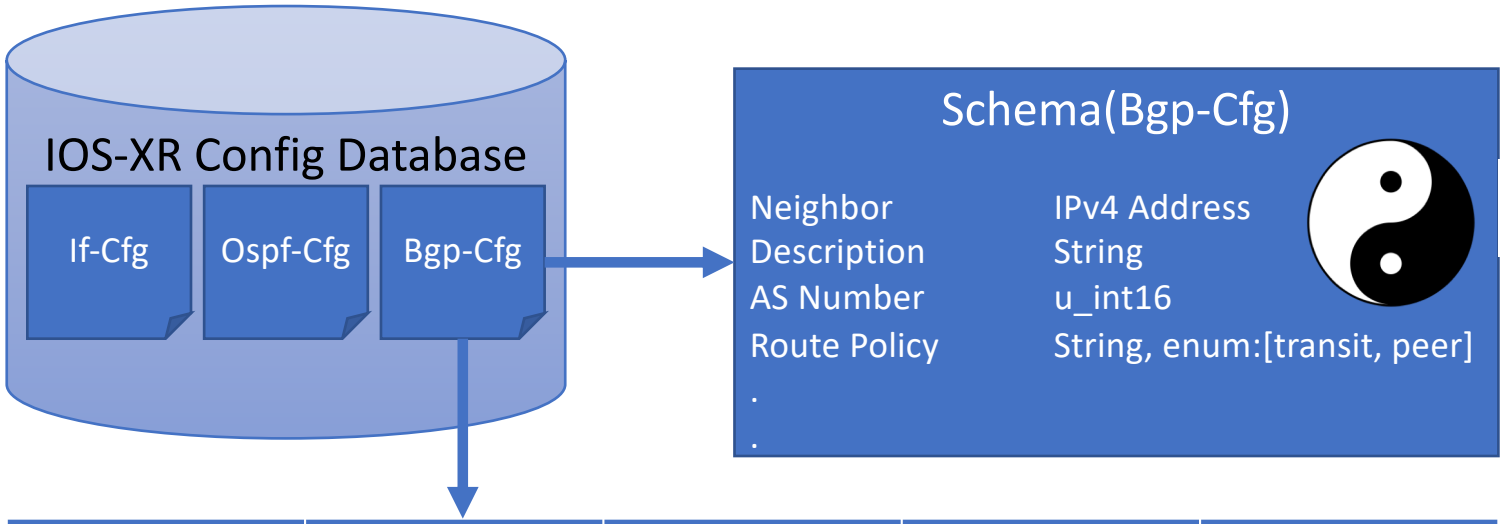
## NetconfもCLIも同じDBを読み書きしている

```
<rpc>
  <get-config>
    <source><running/></source>
  </get-config>
</rpc>
]]>
```



# YANG: NW装置が持つDBのスキーマ

## NW装置の設定文法は、YANGで記述



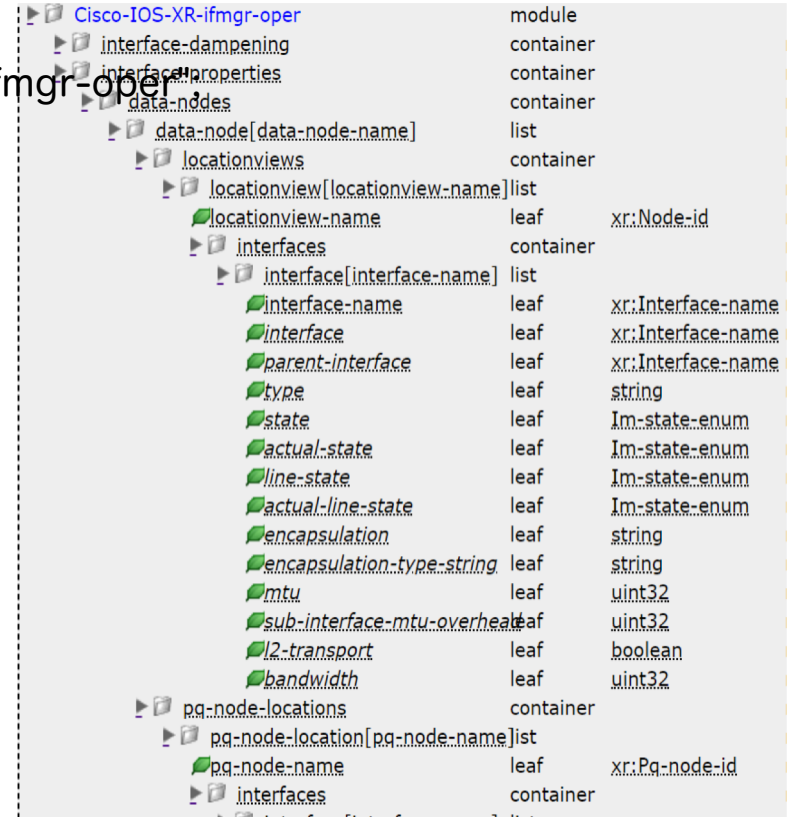

YANGで記述

| Neighbor | Description | AS Number | Route Policy | … |
|---|---|---|---|---|
| 1.2.3.4 | NTT GIN | 2914 | transit | … |
| 9.8.7.6 | Sakura Internet | 7684 | peer | … |
| … | … | … | … | … |

# YANG = NW装置のスキーマ記述言語

## NWベンダーさんが装置の設定スキーマを用意してくれる

```
module Cisco-IOS-XR-ifmgr-oper {
  namespace "http://cisco.com/ns/yang/Cisco-IOS-XR-ifmgr-oper";
  prefix ifmgr-oper;
/SNIP/
 container interface-properties {
    config false;
    description
      "interface properties";
    container data-nodes {
      description
        "Operational data for interfaces";
      list data-node {
        key "data-node-name";
        description
          "The location of a (D)RP in the same LR as the
           interface being queried";
        container locationviews {
          description
```



# クラスをインスタンス化してコーディング

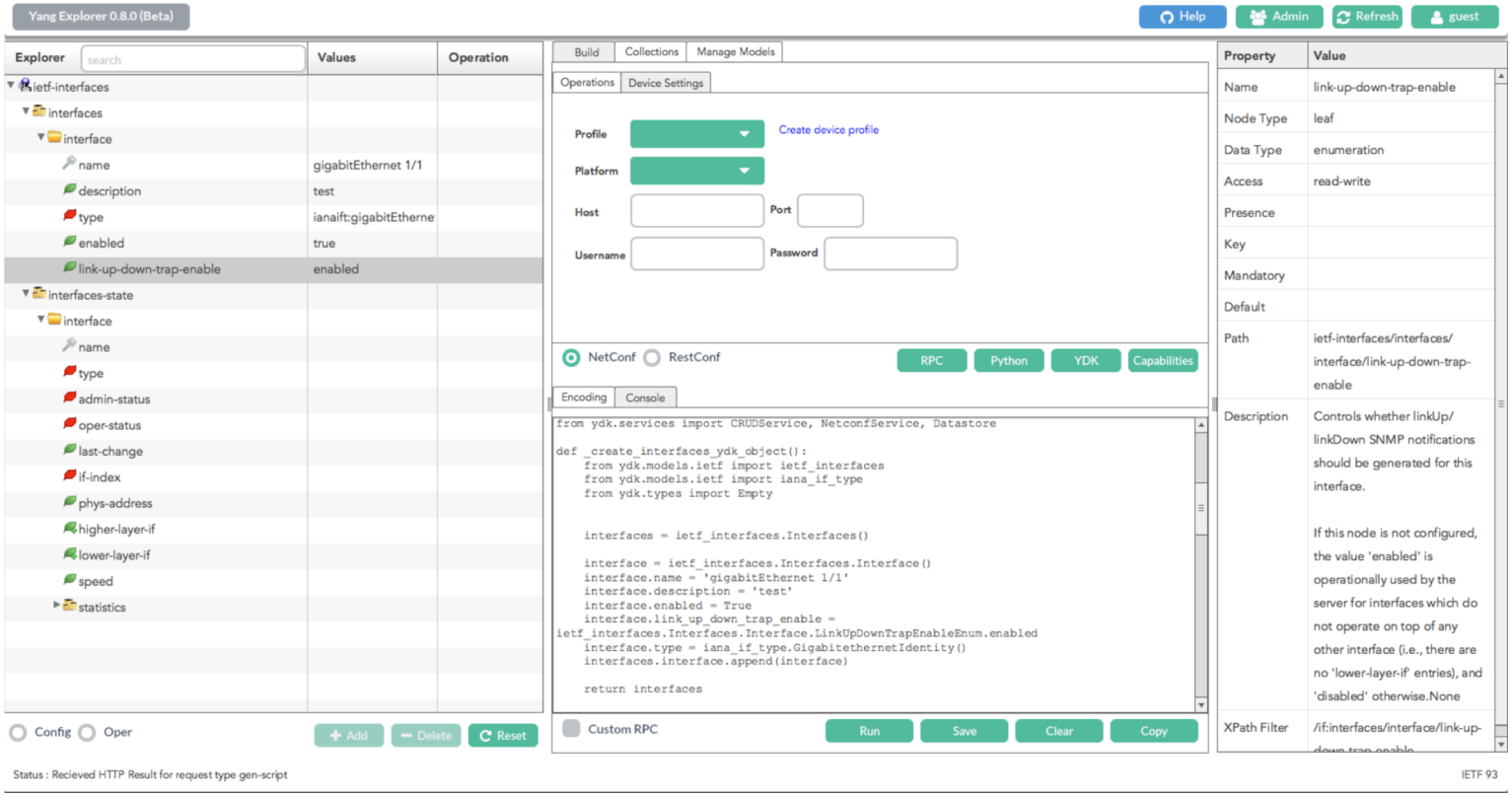
## インスタンスの要素がIDEで自動補完可能！サクサク実装

```
35  from ydk.models.cisco_ios_xr import Cisco_IOS_XR_ipv4_bgp_cfg as xr_bgp
36  from ydk.models.cisco_ios_xr import Cisco_IOS_XR_ipv4_bgp_datatypes as xr_bgp_types
37
38  from session_mgr import init_logging
39
40  def odl_run(crud_service, session):
41
42      bgp = xr_bgp.Bgp()
43      instance = bgp.Instance()
44      bgp.instance.append(instance)
45      instance.instance_name = 'test'
46      instance.ne
47      instance_as
48      instance_as
49      instance.in
50      four_byte_a
51      four_byte_a
52      four_byte_a
53      instance_as
54
55      # global ad
56      global_af =
57      global_af.a
58      global_af.e
59      four_byte_as.default_vrf.global_.global_afs.global_af.append(global_af)
```

```
neighbor
neighbor_address
neighbor_group
neighbor_group_add_member
neighbor_group_af
neighbor_group_afs
neighbor_group_name
neighbor_groups
neighbors
Neighbor
NeighborGroup
NeighborGroupAf
```

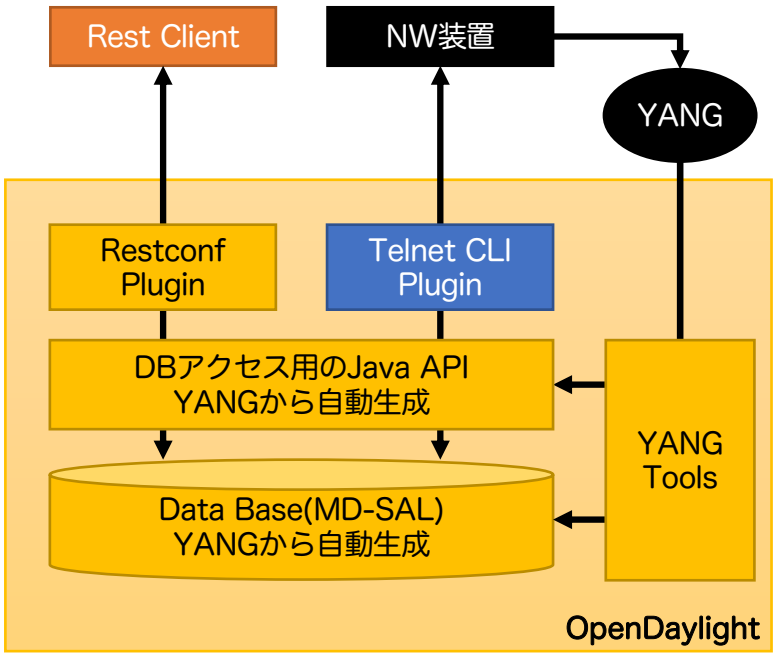
# サンプルコードを自動生成

## YANG-Explorerで簡単サンプルコード生成



# Telnet CLI装置対応の例

## CLIだけ対応装置、非CLI対応装置もYANGで違いを吸収




- まずは後付でYANGを定義する
  - 値を入れる器を用意してやる
  - 当然、型の定義は必要
  - OpenConfigなんかで代用もあり
- 定義したYANGスキーマをインポート
  - スキーマに合致したDBとアクセス用APIがYANGから自動生成
  - この時点でRestconfでNBIアクセス可能に
- Southbound Pluginを実装する
  - トランスポートをプロトコルに合わせて実装する
  - 構造化されていないテキストからYANGスキーマに合致したデータ構造に変換を実装する
  - 正規表現の処理をココに局所化する